

Decentralized Voting System on Blockchain: TrustVote

1. Introduction

Electronic voting systems require mechanisms for maintaining the integrity of ballots, controlling voter eligibility, enforcing election rules, and producing verifiable results. Traditional voting systems are often centralised, meaning that critical aspects of the voting process, such as the count and control mechanisms, are entrusted to a small number of trusted entities. The project thus investigates the use of blockchain technology to build a prototype of a voting system in which key actions in the election process are executed according to smart contract logic and voting history is stored as smart contract state. The system aims to achieve security, transparency, accessibility, efficiency and trust.

The developed system is an educational prototype of blockchain-based voting system (TrustVote) that has been put together using Solidity, Web3.js, MetaMask, Truffle, and Ganache. The current implementation is concerned with on chain election-state management, checking voter eligibility, checking candidate management, controlled vote-casting, checking duplicate voting from the same registered address, and getting the results of elections from the smart-contract state.

2. Problem Statement

Traditional voting systems can include centralised databases, intermediaries, manual counting, and restricted ability to count votes directly. Project documentation raises concerns such as lack of transparency, fraud or tampering, slow results, cost of operation, centralisation and accessibility issues.

The technical issue that TrustVote will address is:

What could the use of blockchain and smart-contract technology be in the event of implementing a voting mechanism where the rules of voting and the recording of votes are carried out by verifiable program logic, in addition to the application controls?

3. Objectives

The main objectives are:

1. To propose a prototype for voting based on blockchain and smart-contract technology that is compatible with Ethereum.
2. To store the candidate and voting data as blockchain state.
3. To limit administrative tasks to the specified admin account.
4. To register eligible voters prior to the polling.
5. To make sure that an eligible blockchain address can only vote once.
6. To programatically enforce the election lifecycle.
7. To deliver vote tallies and transaction records on chain in a trustless manner.
8. To test the smart contract's functional and security-related behaviour.

These goals determine the capabilities and security needs of the TrustVote system.

4. Existing System

Traditional centralized voting systems rely on vote records, result processing, voter information and election configuration to be held within a central server or controlled database. This establishes a central point of trust: administrators and infrastructure of a central system become a very significant component of the integrity of the system.

The project highlights a number of challenges to these approaches, such as:

- limited transparency in vote counting;
- susceptibility to fraud or tampering;
- centralised control;
- operational cost;
- delayed counting and result publication;
- accessibility limitations.

TrustVote explores the possibility of solving some of these issues by transferring some of the key rules of the election and changes in the voting state into a smart contract.

5. Proposed TrustVote System

TrustVote is implemented as a blockchain-connected web application.

A) The current system architecture can be represented as:

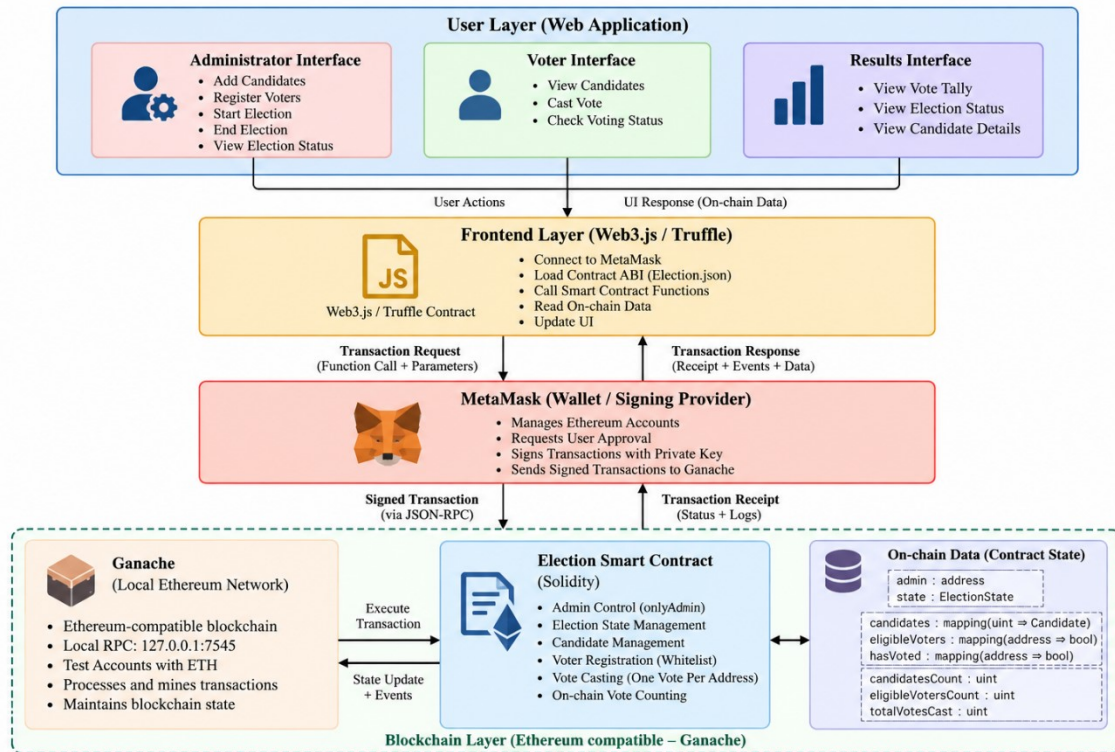
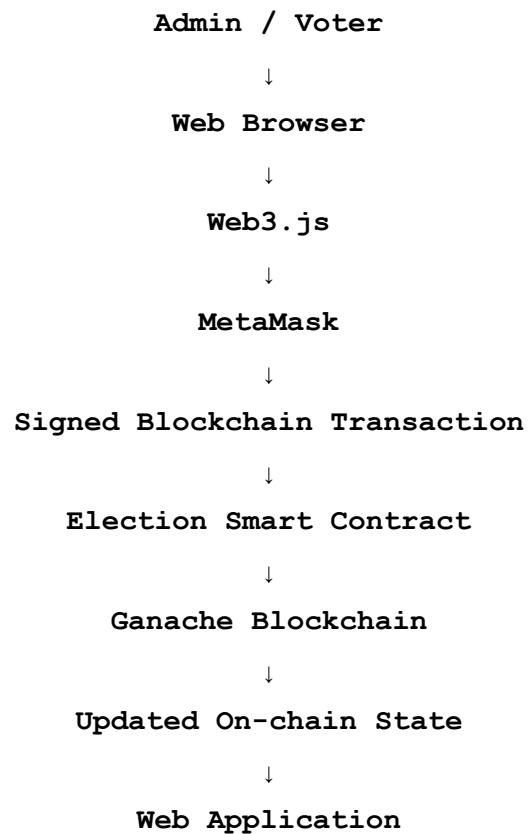


Figure 1: System Architecture of TrustVote

The browser connects to blockchain via Web3.js and interacts with the blockchain using MetaMask to create and submit state-changing transactions. Security critical validation is done by the smart contract itself.

B) The operational flow is:



The project uses Ganache as a local blockchain (Based on Ethereum) for development and testing rather than deploying the prototype on the public Ethereum mainnet.

6. Blockchain

A blockchain is a distributed ledger, and each transaction is recorded on the blockchain and linked to one another through cryptographic technique. In TrustVote, blockchain offers a transaction-based ledger, where accepted changes in the smart-contract state are added to the blockchain state.

TrustVote uses blockchain to serve as the execution environment for the election smart contract, and also to store all the information required for the election.

So, the project focuses on blockchain primarily for the following purposes:

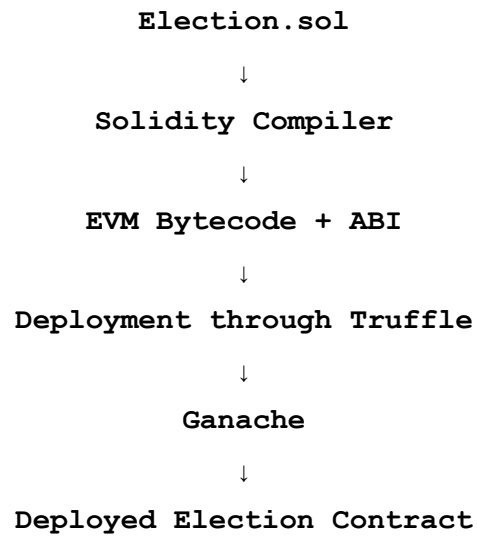
1. maintaining election state;
2. storing candidate information;
3. keeping a voter's eligibility state; and
4. keeping track of state changes that are related to a vote;
5. maintaining transaction history;
6. retrieving the final tally from contract state.

7. Ethereum

TrustVote is based on the smart-contract model on Ethereum.

Ethereum offers a platform for deploying and running programmable contracts. In this project, the contract is written on Solidity and then compiled to an EVM compatible bytecode.

The development process is:



8. Smart Contract

The core logic is implemented in:

contracts/Election.sol

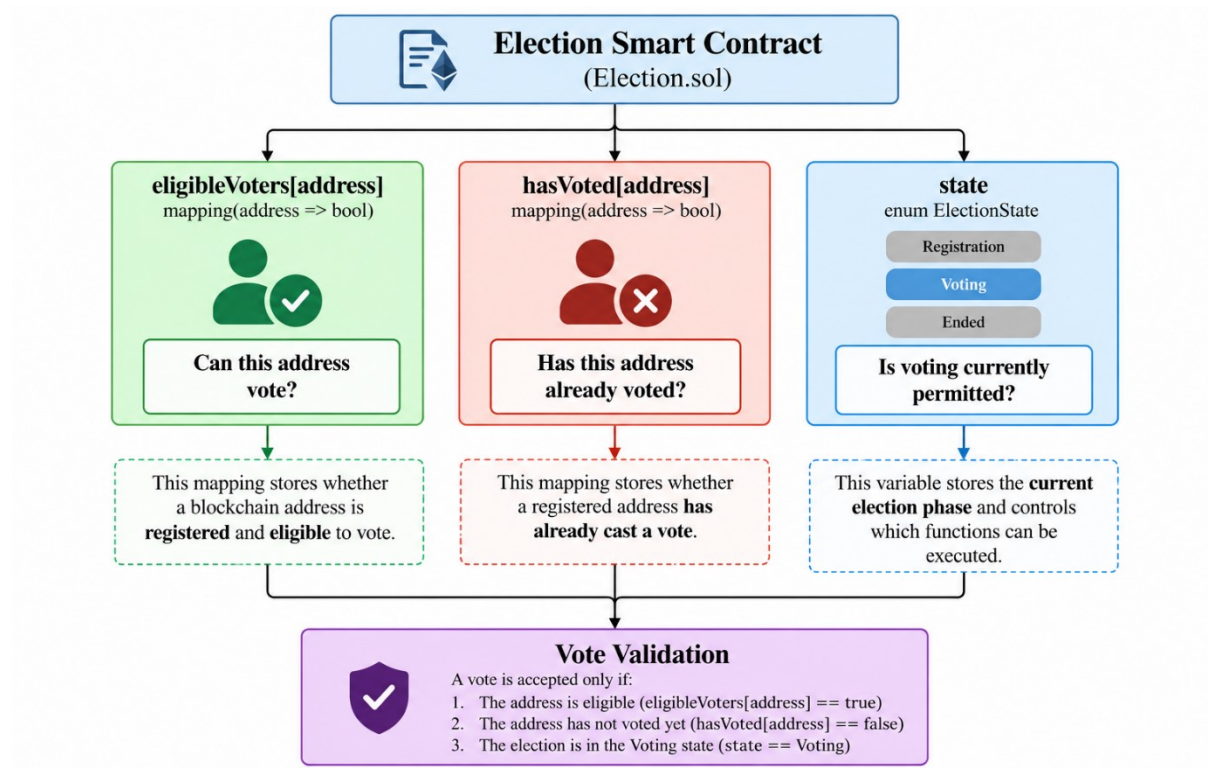


Fig. 2. Key Smart-Contract State Variables and Their Purpose in TrustVote

The contract separates three major concepts, which is central to the security model.

The contract maintains:

```
enum ElectionState {  
    Registration,  
    Voting,  
    Ended  
}
```

It also maintains:

```
address public admin;
```

```
mapping(uint => Candidate) public candidates;
```

```
mapping(address => bool) public eligibleVoters;  
mapping(address => bool) public hasVoted;
```

```
uint public candidatesCount;  
uint public eligibleVotersCount;  
uint public totalVotesCast;
```

10. Voter Registration

Voter eligibility is controlled by the administrator.

The smart contract uses:

```
mapping(address => bool) public eligibleVoters;
```

The administrator calls:

```
registerVoter(address _voter)
```

The contract verifies:

```
onlyAdmin
```

and:

```
state == ElectionState.Registration
```

It also rejects:

- the zero address;
- duplicate registration.

After successful registration:

```
eligibleVoters[_voter] = true;  
eligibleVotersCount++;
```

The important design principle is:

Possessing a blockchain address does not automatically make an address eligible to vote.
Eligibility is explicitly established by the election administrator.

11. Candidate Management

Candidates are stored through:

```
struct Candidate {  
    uint id;  
    string name;  
    string party;  
    uint voteCount;  
}
```

and:

```
mapping(uint => Candidate) public candidates;
```

The administrator invokes:

```
addCandidate(string memory _name, string memory _party)
```

The contract checks:

```
Caller = Admin
```

```
AND
```

```
Election state = Registration
```

```
AND
```

```
Candidate name ≠ empty
```

```
AND
```

```
Party ≠ empty
```

A candidate receives a sequential identifier:

```
CandidateID=CandidateCount
```

and begins with:

```
VoteCount=0
```

Candidate registration is therefore completed before the election enters the Voting state.

12. Election Management

TrustVote implements a controlled lifecycle:

Registration



Voting



Ended

The administrator starts the election using:

```
startElection()
```

The contract requires:

```
candidatesCount >= 2
```

and changes:

```
state = ElectionState.Voting;
```

The election is closed through:

```
endElection()
```

which changes:

```
state = ElectionState.Ended;
```

The state transitions are intentionally one-directional. Once the election reaches Ended, it cannot be restarted.

13. Voting Algorithm

The core voting function is:

```
function vote(uint _candidateId)
```

The algorithm is:

1. Check election state.
2. Check voter eligibility.
3. Check whether voter has already voted.
4. Validate candidate ID.
5. Mark voter as having voted.
6. Increment candidate's vote count.
7. Increment total vote count.
8. Emit VoteCast event.

In logical form:

$$VoteAllowed = VotingState \wedge Eligible(v) \wedge \neg HasVoted(v) \wedge ValidCandidate(c)$$

Where:

- $VotingState$ = the election is currently in the Voting state.
- $Eligible(v)$ = voter v is registered/eligible.
- $HasVoted(v)$ = voter v has already voted.
- $ValidCandidate(c)$ = candidate c is valid.
- $\neg$ = NOT.

Only when all conditions are true is the vote accepted.

After acceptance:

$$\begin{aligned}HasVoted(v) &= True \\VoteCount(c) &= VoteCount(c) + 1 \\TotalVotes &= TotalVotes + 1\end{aligned}$$

This prevents the same registered address from successfully executing the voting function more than once.

14. MetaMask / Web3.js

The frontend logic is mainly implemented in:

```
src/js/app.js
```

MetaMask exposes:

```
window.ethereum
```

The application requests the user's account through:

```
window.ethereum.request({  
  method: 'eth_requestAccounts'  
})
```

The selected address becomes:

```
App.account
```

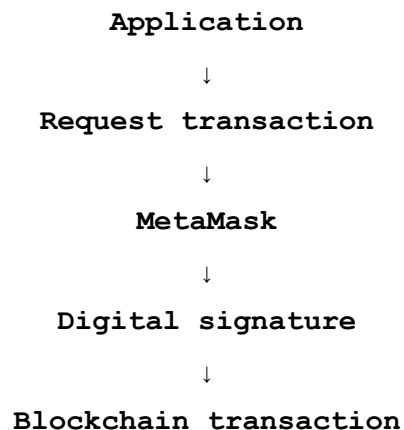
Web3.js, together with the Truffle contract abstraction, is used by the frontend to communicate with the deployed smart contract.

For example, the voting operation follows this conceptual path:

```
instance.vote(candidateId, {  
  from: App.account  
});
```

This causes MetaMask to request transaction approval.

Therefore:



MetaMask is consequently responsible for control of the account's signing authority, while the smart contract is responsible for validating the operation. Web3.js is the blockchain-interaction layer and MetaMask is the wallet/blockchain interface.

15. Ganache

The local blockchain environment is configured in:

truffle-config.js

with:

- host = 127.0.0.1
- port = 7545

Ganache provides:

- test Ethereum accounts;
- test ETH;
- blocks;
- transactions;
- RPC connectivity;

a controlled environment for smart-contract testing.

Ganache provides a controlled local blockchain environment for developing, deploying, and testing the smart contract.

16. Security Mechanisms

The principal security mechanisms are implemented at the smart-contract layer.

- **Administrative access control**

```
modifier onlyAdmin()
```

checks:

```
msg.sender == admin
```

- **Election-state control**

```
modifier inState(ElectionState _expectedState)
```

prevents functions from being executed in an inappropriate election phase.

- **Voter eligibility**

```
eligibleVoters[msg.sender]
```

must be true before voting.

- **Double-voting prevention**

```
!hasVoted[msg.sender]
```

must be true.

- **After voting:**

```
hasVoted[msg.sender] = true;
```

- **Candidate validation**

```
_candidateId > 0 &&  
_candidateId <= candidatesCount
```

prevents invalid candidate references.

- **Event logging**

The contract emits:

CandidateAdded

VoterRegistered

ElectionStarted

ElectionEnded

VoteCast

These events provide transaction-level activity logs.

17. Testing

The project uses a dedicated Solidity/Truffle test suite:

test/election.js

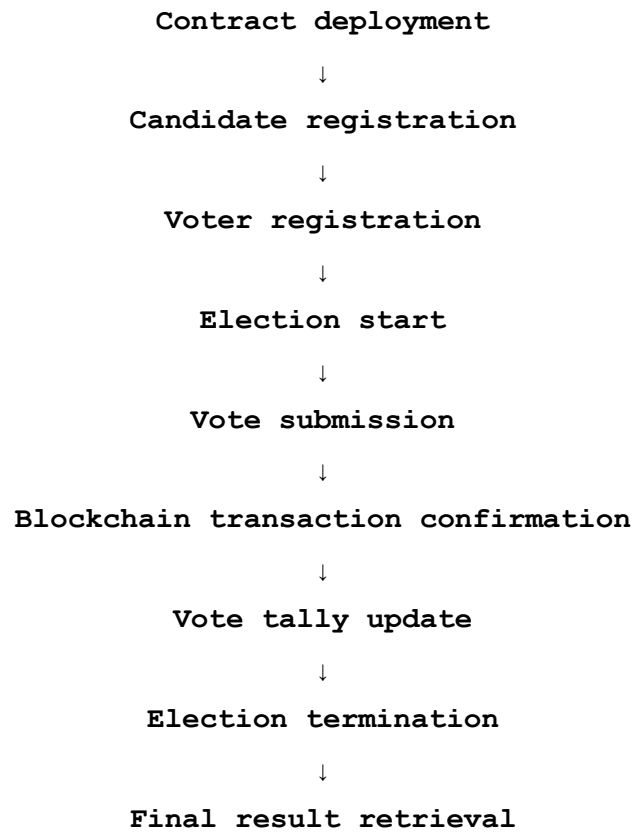
There are 21 test cases in the current source for deployment, administrative access control, candidate registration, voter registration, election lifecycle, vote validation, duplicate-vote prevention, and post-election restrictions.

Selected security and boundary-condition tests include:

Test	Purpose
ST-03	Non-admin cannot add candidates
ST-06	Non-admin cannot register voters
ST-09	Non-admin cannot start election
ST-11	Voting before election starts is rejected
ST-13	Candidate/voter modification after start is rejected
ST-14	Ineligible voter is rejected
ST-16	Double voting is rejected
ST-17	Invalid candidate ID is rejected
ST-18	Non-admin cannot end election
ST-20	Voting after election ends is rejected
ST-21	Election cannot be restarted or moved backwards

18. Results

The functional prototype demonstrates the complete election lifecycle:



The execution of the system successfully connected with the MetaMask account, registered the candidates, whitelisted the voters, activated the election, signed the transactions, recorded the votes, and finally presented the results.

Final result is computed in the smart-contract state, not by presentation layer. The presentation layer does not calculate the final tally but gets it from the smart-contract state.

19. Limitations

TrustVote is an educational prototype, not a production-ready national election platform.

The major limitations are:

1. Voter identity

A blockchain address represents an account, not necessarily a unique real-world person. Therefore:

One Address $\neq$ One Human

The whitelist prevents unauthorised addresses from voting, but it does not by itself solve real-world identity verification or Sybil resistance.

2. Ballot secrecy

The current contract records:

- voter address
- candidate ID
- vote state

Through blockchain transactions/events. Therefore, the current design should not be described as a fully anonymous or secret-ballot system.

3. Local blockchain

The prototype is running on Ganache, and is not an example of the characteristics of a public production Ethereum deployment.

4. Scalability

There has been no large scale load experiment that has determined performance for thousands or millions of concurrent voters.

5. Endpoint and wallet security.

Blockchain-level controls cannot prevent misuse of a compromised user wallet or private key.

If an attacker gains control of a voter's private key, the attacker may be able to act as that blockchain account.

20. Future Scope

The prototype can be extended in four main directions in future work:

- Cryptographic voting methods that allow one to vote while preserving voters' privacy with respect to ballot choices.
- More robust mechanisms of voter identity to tie blockchain eligibility with verified real world identity.
- Static analysis, fuzzer, property based testing and formal verification of smart-contracts.
- Evaluation of performance and scalability with controlled increments of transaction loads.

21. Conclusion

TrustVote illustrates the use of blockchain and smart-contract technology for the development of a controlled electronic voting prototype. Importing crucial elements of the election process into a smart contract on Solidity, and introducing blockchain state for handling candidates, voter eligibility, election lifecycle, vote counting, and transaction recording.

The key security aspect of the system is that the smart contract will perform the security critical validation and not rely on the user interface alone. Administrative authority, eligibility of voters, election-state restrictions, validation of candidates, and One Vote Per Registered Address are thus directly coded as programmable contracts.

Beyond its specific use for voting, the prototype lays the groundwork for future investigations on privacy-preserving voting, identity assurance, formal smart-contract verification, and scalability. It should thus be interpreted as a research and educational prototype of an election control system on the blockchain and a verifiable state management system, not as a finished production election system.

20. References

- [1] M.-V. Vladucu, Z. Dong, J. Medina, and R. Rojas-Cessa, "E-Voting Meets Blockchain: A Survey," *IEEE Access*, vol. 11, pp. 23293–23308, 2023, doi: 10.1109/ACCESS.2023.3253682.
- [2] M. Hajian Berenjestanaki, H. R. Barzegar, N. El Ioini, and C. Pahl, "Blockchain-Based E-Voting Systems: A Technology Review," *Electronics*, vol. 13, no. 1, art. 17, 2024, doi: 10.3390/electronics13010017.

